Second Year Engineering

23CSE3004-Operating System

Class - S.Y. (SEM-II)

Unit - II

PROCESS SYNCHRONIZATION AND DEADLOCKS

AY 2025-2026 **SEM-II**

Unit-II Syllabus

Process Synchronization: Cooperating processes and Race Conditions, the critical-section problem, Peterson's solution, Semaphores, Classic problems of synchronization (Producer-Consumer Problem, Dining-Philosophers Problem, Readers and Writers Problem, Sleeping Barber Problem), Inter process Communication: Overview of IPC, Examples of IPC Systems, Communication in Client Server Systems

Deadlock: System Model, Deadlock characterization, Deadlock and Starvation, Methods for Handling Deadlocks, Resource Allocation Graphs, Deadlock Prevention, Deadlock Avoidance, Deadlock Detection, Recovery From Deadlock.

Case study: Bankers algorithm

Unit –II

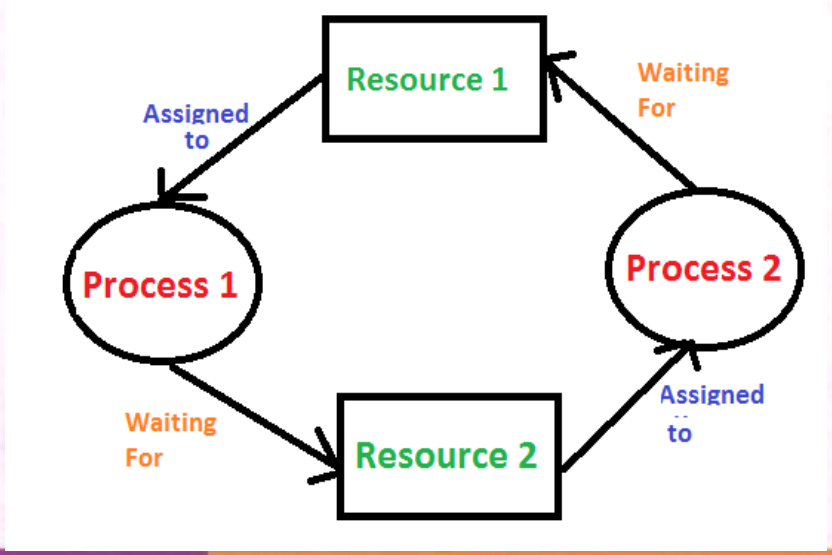
Process Synchronization is used in a computer system to ensure that multiple processes or threads can run concurrently without interfering with each other.

The main objective of process synchronization is to ensure that multiple processes **access shared resources without interfering with each other and to prevent the possibility of inconsistent data due to concurrent access.**

Cont..

In a multi-process system, synchronization is necessary to ensure **data consistency and integrity,** and to avoid the risk of **deadlocks** and other synchronization problems.

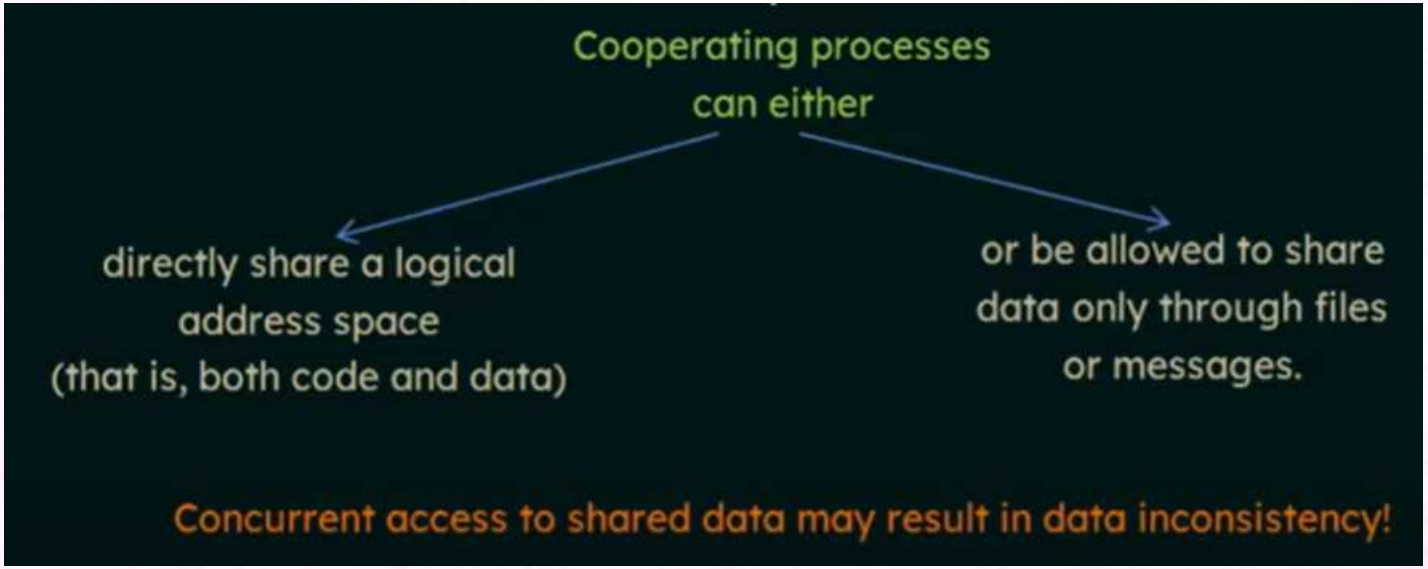
To achieve this, various synchronization techniques such as semaphores, monitors, and critical sections are used.



On the basis of synchronization, processes are categorized as one of the following two types:

- **Independent Process**: The execution of one process does not affect the execution of other processes.

- **Cooperative Process**: A process that can affect or be affected by other processes executing in the system.



>>ps/grep "chrome"/wc

From this example, we can understand the concept of cooperative processes, where some processes produce and others consume, and thus work together. This type of problem must be handled by the operating system, as it is the manager.

Conditions That Require Process Synchronization

1.Critical Section: It is that part of the program where shared resources are accessed. Only one process can execute the critical section at a given point of time. If there are no shared resources, then no need of synchronization mechanisms.

2.Race Condition: It is a situation wherein processes are trying to access the critical section and the final result depends on the order in which they finish their update. Process Synchronization mechanism need to ensure that instructions are being executed in a required order only.

3.Pre Emption: Preemption is when the operating system stops a running process to give the CPU to another process. This allows the system to make sure that important tasks get enough CPU time. This is important as mainly issues arise when a process has not finished its job on shared resource and got preempted. The other process might end up reading an inconsistent value if process synchronization is not done.

A race condition in an operating system (OS) is a software bug that occurs when multiple processes or threads access the same resource at the same time. The order in which the processes execute is unpredictable, which can lead to unexpected results

```
P0
{
Read(A);
A=A+1;
Write(A);
}
```

```
P1
{
Read(A);
A=A+1;
Write(A);
}
```

The order of execution of the processes is P0, P1 respectively. Consider that process P0 has read the variable A=0. Suddenly context switch happens and P1 takes the charge, and start executing. P1 would increment the value of A (A=1). After execution P1 gives the charge again to P0.

Now the value of A for P0 is 0 & when it starts executing it would increment the value of A, from 0 to 1.

So here, when both the processes P0 & P1 end up accessing the variable A, the value of A =1 which is different from the value of A=2

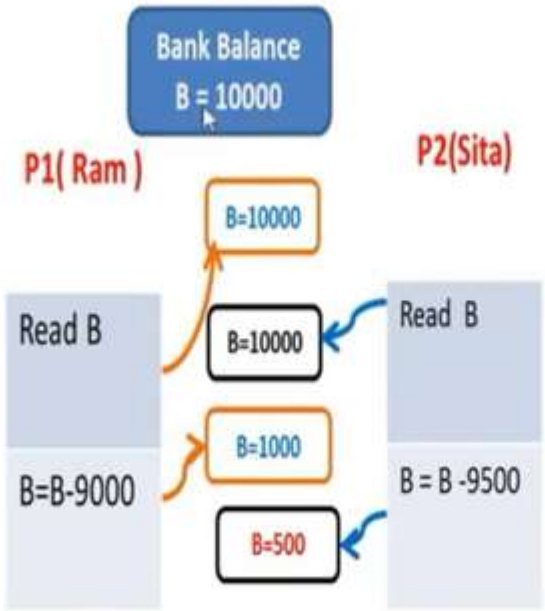
Now, this type of condition where the sequence of execution of the processes affects the result is called race condition.

To avoid a race condition, the process should access the common resources under its critical section.

Example 1 Joint Bank Account

Sample C program: Joint bank account with multiple account holders (P1 and P2)

```
main()
{
 statement 1
 statement 2
  .........
 B=10000 /*Bank Balance */
  .....
 statement n
}
```



Possible Situations

case 1 : Deadlock
case 2 : P1 (B) , P2(B) , P1(B=B -9000), P2(B=B-9500)
Case 3 : P2 (B) , P1(B) , P2(B=B -9000), P1(B=B-9500)

case 3 : P1 and P2
case 4 : P2 and P1

Banking, Database access, Inventory system File access , printer access, Restaurant Kitchen, Cash Register in a Store:

The Critical-Section Problem

Consider a system consisting of n processes { $P_0, P_1, \ldots, P_n$ }.

Each process has a segment of code, called a

critical section

in which the process may be changing common variables, updating a table, writing a file, and so on.

When one process is executing in its critical section, no other process is to be allowed to execute in its critical section.

That is, no two processes are executing in their critical sections at the same time.

The critical-section problem is to design a protocol that the processes can use to cooperate.

- Each process must request permission to enter its critical section.
- The section of code implementing this request is the entry section.
- The critical section may be followed by an exit section.
- The remaining code is the remainder section.

```
do {
        entry section
            critical section
        exit section
            remainder section
    } while (TRUE);
```

Figure: General structure of a typical process.

ESO ACADEMY

A solution to the critical-section problem must satisfy the following three requirements:

1. Mutual exclusion:

If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.

2. Progress:

If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in the decision on which will enter its critical section next, and this selection cannot be postponed indefinitely.

3. Bounded waiting:

There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

```
P0
{
Read(A);
A=A+1;
Write(A);
}
```

```
P1
{
Read(A);
A=A+1;
Write(A);
}
```

```
P0
While(1)
{
While(turn!=0);
Critical Section;
turn=1;
Remainder Section;
}
```

```
P1
While(1)
{
While(turn!=1);
Critical Section;
turn=0;
Remainder Section;
}
```

Banking, Database access, Inventory system
File access , printer access, Restaurant Kitchen, Cash Register in a Store:

Peterson's solution

What is Peterson's Algorithm?

Peterson's Algorithm is a well-known solution for ensuring mutual exclusion in process synchronization. It is designed to manage access to shared resources between two processes in a way that prevents conflicts or data corruption.

The algorithm ensures that only one process can enter the critical section at any given time while the other process waits its turn. Peterson's Algorithm uses two simple variables one to indicate whose turn it is to access the critical section and another to show if a process is ready to enter.

This method is often used in scenarios where two processes need to share resources or data without interfering with each other. It is simple, easy to understand, and serves as a foundational concept in process synchronization.

Peterson's Solution

- A classic software-based solution to the critical-section problem.
- May not work correctly on modern computer architectures.
- However, it provides a good algorithmic description of solving the critical-section problem and illustrates some of the complexities involved in designing software that addresses the requirements of mutual exclusion, progress, and bounded waiting requirements.

Peterson's solution is restricted to two processes that alternate execution between their critical sections and remainder sections. Let's call the processes P_i and P_j

Peterson's solution requires two data items to be shared between the two processes:

int turn

→ Indicates whose turn it is to enter its critical section.

boolean flag [2]

→ Used to indicate if a process is ready to enter its critical section.

int turn

Indicates whose turn it is to enter its critical section.

boolean flag [2]

Used to indicate if a process is ready to enter its critical section.

Structure of process P_i in Peterson's solution

```
do {
    flag [i] = true ;
    turn = j ;
    while ( flag [ j ] && turn == [ j ] ) ;

    critical section

    flag [i] = false ;

    remainder section

} while (TRUE) ;
```

Structure of process P_j in Peterson's solution

```
do {
    flag [ j ] = true ;
    turn = i ;
    while ( flag [ i ] && turn == [ i ] ) ;

    critical section

    flag [ j ] = false ;

    remainder section

} while (TRUE) ;
```

Semaphores

- Semaphore proposed by Edsger Dijkstra, is a technique to manage concurrent processes by using a simple integer value, which is known as a semaphore.
- Semaphore is simply a variable which is non-negative and shared between threads. This variable is used to solve the critical section problem and to achieve process synchronization in the multiprocessing environment.
- A semaphore S is an integer variable that, apart from initialization, is accessed only through two standard atomic operations: wait ( ) and signal ( ).

wait ( ) → P [from the Dutch word proberen, which means "to test"]

signal ( ) → V [from the Dutch word verhogen, which means " to increment"]

Semaphores

are used in operating systems to control access to shared resources and coordinate the execution of multiple processes. They are a type of synchronization primitive that help prevent race conditions and data corruption.

- **Prevent race conditions**: Semaphores ensure that only one process can access a shared resource at a time.
- **Manage memory resources**: Semaphores can be used to manage memory resources, such as files or databases.
- **Protect against data corruption**: Semaphores help to prevent the data corruption or inconsistency.
- **Ensure process synchronization**: Semaphores helps to coordinate the execution of multiple processes or threads.

wait ( ) → P [from the Dutch word proberen, which means "to test"]

signal ( ) → V [from the Dutch word verhogen, which means " to increment"]

Definition of wait ( ):

```
P (Semaphore S) {
    while (S <= 0)
      ; // no operation
    S-- ;
}
```

Definition of signal ( ):

```
V (Semaphore S) {
    S++ ;
}
```

All the modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Definition of wait ( ):

```
P (Semaphore S) {
    while (S <= 0)
      ; // no operation
    S-- ;
}
```

Definition of signal ( ):

```
V (Semaphore S) {
    S++ ;
}
```

All the modifications to the integer value of the semaphore in the wait () and signal() operations must be executed indivisibly. That is, when one process modifies the semaphore value, no other process can simultaneously modify that same semaphore value.

Types of Semaphores:

1. Binary Semaphore:

The value of a binary semaphore can range only between 0 and 1. On some systems, binary semaphores are known as mutex locks, as they are locks that provide mutual exclusion.

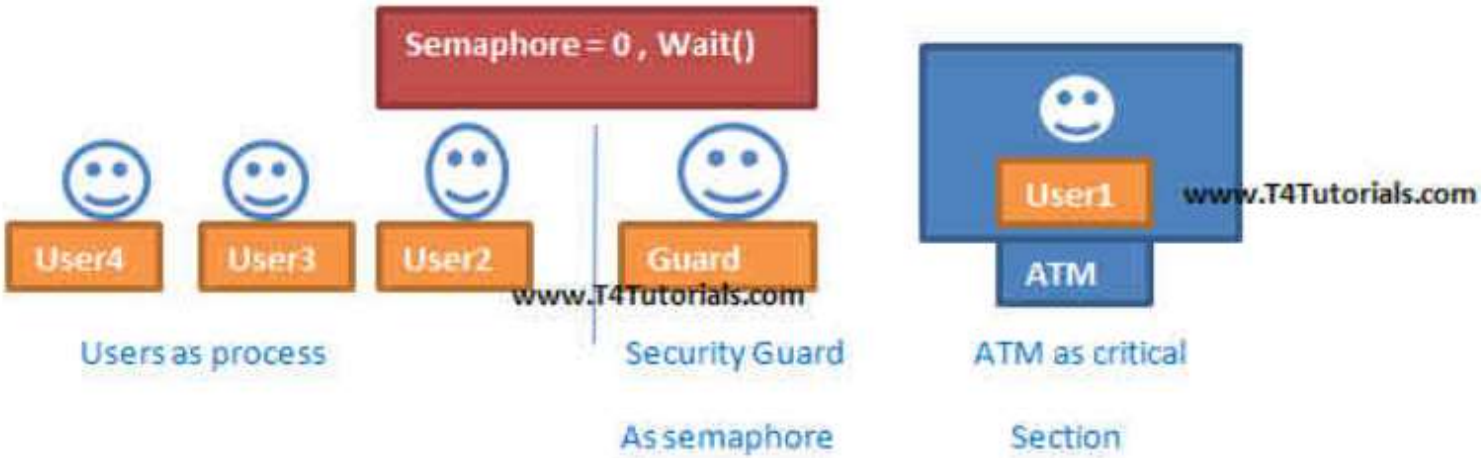
2. Counting Semaphore:

Its value can range over an unrestricted domain. It is used to control access to a resource that has multiple instances.

NESO ACADEMY

Disadvantages of Semaphores

- The main disadvantage of the semaphore definition that was discussed is that it requires busy waiting.
- While a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code.
- Busy waiting wastes CPU cycles that some other process might be able to use productively.
- This type of semaphore is also called a spinlock because the process "spins" while waiting for the lock.



User1(Process1) is using ATM(Critical section), Guard (Semaphore) is performing wait operation and stops the user2, user3 and user4

Suppose you had 5 computers which share 2 printers and they all tried to print at the same time, then obviously error. but if we set the number of semaphore to 2(i.e initial number of resources of printer) and every time a computer tries to access the printer the semaphore value decreases and increases the semaphore value when it releases. So when the first computer tries to access the printer semaphore value decreases to 1 and the 1st printer is allocated to the first pc and when the second computer tries to access,semaphore value decreases to 0 and second printer is allocated to 2nd pc ,now when the third computer to tries to access the printer since the semaphore value is 0,no printer is allocated and the requesting pc would have to wait until another pc would release a printer(in that case the semaphore value increases),thus all pc's work without interrupting other pc's work.

Criteria	Binary Semaphore	Counting Semaphore
Definition	A Binary Semaphore is a semaphore whose integer value range over 0 and 1.	A counting semaphore is a semaphore that has multiple values of the counter. The value can range over an unrestricted domain.

Structure Implementation	typedef struct {
	int semaphore_variable;
	}binary_semaphore;
		typedef struct {
		int semaphore_variable;
		Queue list;
		//A queue to store the list of task
		}counting_semaphore;

Representation	0 means that a process or a thread is accessing the critical section, other process should wait for it to exit the critical section. 1 represents the critical section is free.	The value can range from 0 to N, where N is the number of process or thread that has to enter the critical section.

Mutual Exclusion	Yes, it guarantees mutual exclusion, since just one process or thread can enter the critical section at a time.	No, it doesn't guarantees mutual exclusion, since more than one process or thread can enter the critical section at a time.
Bounded wait	No, it doesn't guarantees bounded wait, as only one process can enter the critical section, and there is no limit on how long the process can exist in the critical section, making another process to starve.	Yes, it guarantees bounded wait, since it maintains a list of all the process or threads, using a queue, and each process or thread get a chance to enter the critical section once. So no question of starvation.
Starvation	No waiting queue is present then FCFS (first come first serve) is not followed so,starvation is possible and busy wait present	Waiting queue is present then FCFS (first come first serve) is followed so,no starvation hence no busy wait.
Number of instance	Used only for a single instance of resource type R.it can be usedonly for 2 processes.	Used for any number of instance of resource of type R.it can be used for any number of processes.

To overcome the need for busy waiting, we can modify the definition of the wait () and signal () semaphore operations.

- When a process executes the wait () operation and finds that the semaphore value is not positive, it must wait.
- However, rather than engaging in busy waiting, the process can block itself.
- The block operation places a process into a waiting queue associated with the semaphore, and the state of the process is switched to the waiting state.
- Then control is transferred to the CPU scheduler, which selects another process to execute.

Deadlocks and Starvation

- The implementation of a semaphore with a waiting queue may result in a situation where two or more processes are waiting indefinitely for an event that can be caused only by one of the waiting processes.
- The event in question is the execution of a signal() operation. When such a state is reached, these processes are said to be deadlocked.

P0
wait (S);
wait (Q);
.
.
.
signal (S);
signal (Q);

P1
wait (Q);
wait (S);
.
.
.
signal (Q);
signal (S);

SO ACADEMY

Classic Problems of Synchronization

(The Bounded-Buffer Problem)

The Bounded Buffer Problem **(Producer Consumer Problem)**, is one of the classic problems of synchronization.

There is a buffer of n slots and each slot is capable of storing one unit of data.

There are two processes running, namely, Producer and Consumer, which are operating on the buffer.

There is a buffer of n slots and each slot is capable of storing one unit of data.

There are two processes running, namely, Producer and Consumer, which are operating on the buffer.

- The producer tries to insert data into an empty slot of the buffer.
- The consumer tries to remove data from a filled slot in the buffer.
- The Producer must not insert data when the buffer is full.
- The Consumer must not remove data when the buffer is empty.
- The Producer and Consumer should not insert and remove data simultaneously.

1. m (mutex), a binary semaphore which is used to acquire and release the lock.
2. empty, a counting semaphore whose initial value is the number of slots in the buffer, since, initially all slots are empty.
3. full, a counting semaphore whose initial value is 0.

Producer

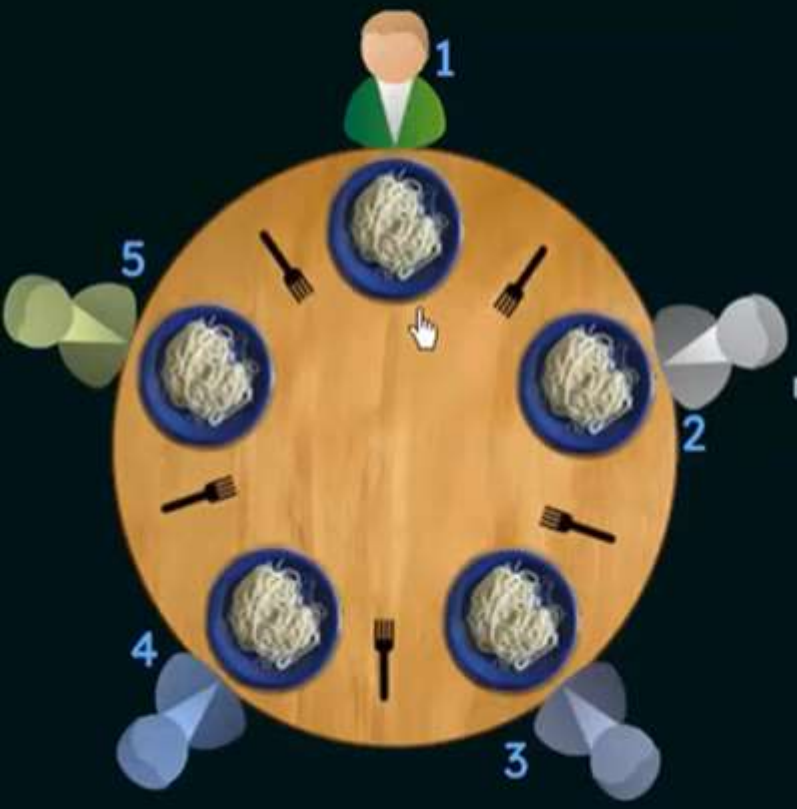
```
do {
  wait (empty); // wait until empty>0
                and then decrement 'empty'
  wait (mutex); // acquire lock
  /* add data to buffer */
  signal (mutex); // release lock
  signal (full);  // increment 'full'
} while(TRUE)
```

Consumer

```
do {
  wait (full); // wait until full>0 and
               then decrement 'full'
  wait (mutex); // acquire lock
  /* remove data from buffer */
  signal (mutex); // release lock
  signal (empty);  // increment 'empty'
} while(TRUE)
```

Classic Problems of Synchronization

(The Dining-Philosophers Problem)



Philosopher either

- **Thinks** → When a philosopher thinks, he does not interact with his colleagues
- **Eats** → When a philosopher gets hungry he tries to pick up the two forks that are closest to him (left & right). A philosopher may pick up only one fork at a time.

One cannot pick up a fork that is already in the hand of a neighbour.

When a hungry philosopher has both his forks at the same time, he eats without releasing his forks. When he has finished eating, he puts down both of his forks and starts thinking again.

One simple solution is to represent each fork/chopstick with a semaphore.

A philosopher tries to grab a fork/chopstick by executing a wait () operation on that semaphore.

He releases his fork/chopsticks by executing the signal () operation on the appropriate semaphores.

Thus, the shared data are

```
semaphore chopstick[5] ;
```

where all the elements of chopstick are initialized to 1.

where all the elements of chopstick are initialized to 1.

The structure of philosopher i

```
do {
wait (chopstick [i] ) ;
wait(chopstick [ (i + 1) % 5] ) ;
. . . .
// eat
signal(chopstick [i]) ;
signal(chopstick [(i + 1) % 5]) ;
// think
}while (TRUE);
```

Although this solution guarantees that no two neighbors are eating simultaneously, it could still create a deadlock.

Suppose that all five philosophers become hungry simultaneously and each grabs their left chopstick. All the elements of chopstick will now be equal to 0.

When each philosopher tries to grab his right chopstick, he will be delayed forever.

Some possible remedies to avoid deadlocks:

- Allow at most four philosophers to be sitting simultaneously at the table.
- Allow a philosopher to pick up his chopsticks only if both chopsticks are available (to do this he must pick them up in a critical section).
- Use an asymmetric solution; that is, an odd philosopher picks up first his left chopstick and then his right chopstick, whereas an even philosopher picks up his right chopstick and then his left chopstick.

Classic Problems of Synchronization

(The Readers-Writers Problem)

- A database is to be shared among several concurrent processes.
- Some of these processes may want only to read the database, whereas others may want to update (that is, to read and write) the database.
- We distinguish between these two types of processes by referring to the former as **Readers** and to the latter as **Writers.**
- Obviously, if two readers access the shared data simultaneously, no adverse affects will result.
- However, if a writer and some other thread (either a reader or a writer) access the database simultaneously, chaos may ensue.

To ensure that these difficulties do not arise, we require that the writers have exclusive access to the shared database.

This synchronization problem is referred to as the readers-writers problem.

Solution to the Readers-Writers Problem using Semaphores:

We will make use of two semaphores and an integer variable:

1. mutex, a semaphore (initialized to 1) which is used to ensure mutual exclusion when readcount is updated i.e. when any reader enters or exit from the critical section.
2. wrt, a semaphore (initialized to 1) common to both reader and writer processes.
3. readcount, an integer variable (initialized to 0) that keeps track of how many processes are currently reading the object.

Writer Process

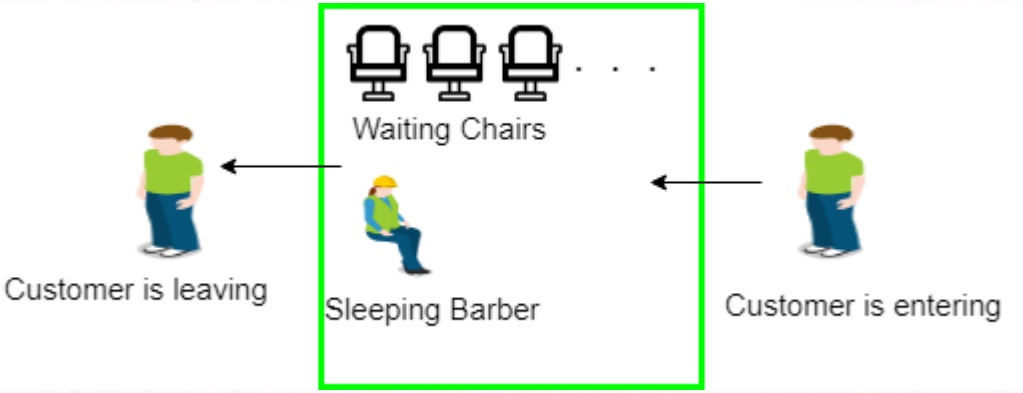
```
do {
/* writer requests for critical
section */
   wait(wrt);
  /* performs the write  */
   // leaves the critical section
   signal(wrt);
} while(true);
```

Reader Process

```
do {
  wait (mutex);
 readcnt++;  // The number of readers has now increased by 1
 if (readcnt==1)
    wait (wrt); // this ensure no writer can enter if there is even one reader
   signal (mutex);  // other readers can enter while this current reader is
                                inside the critical section
 /* current reader performs reading here */
  wait (mutex);
  readcnt--; // a reader wants to leave
  if (readcnt == 0)      //no reader is left in the critical section
     signal (wrt);       // writers can enter
     signal (mutex);  // reader leaves
} while(true);
```

Sleeping Barber problem in Process Synchronization

The problem is to write a program that coordinates the actions of the customers and the barber in a way that avoids synchronization problems, such as deadlock or starvation.



Deadlock

Occurs when a set of processes is stalled because each process is holding a resource and waiting for another process to acquire another resource. In the diagram below, for example, Process 1 is holding Resource 1 while Process 2 acquires Resource 2, and Process 2 is waiting for Resource 1.

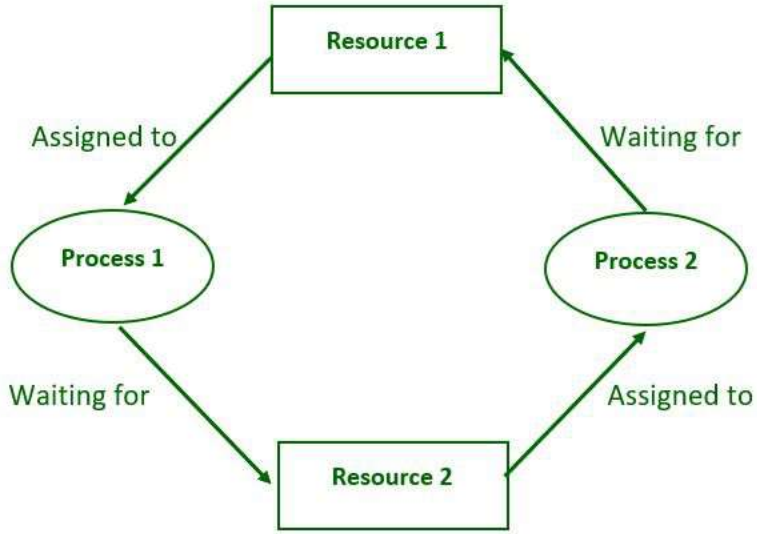


Figure: Deadlock in Operating system

Operations :

In normal operation, a process must request a resource before using it and release it when finished, as shown below.

Request

If the request cannot be granted immediately, the process must wait until the resource(s) required to become available. The system, for example, uses the functions open(), malloc(), new(), and request ().

Use –

The process makes use of the resource, such as printing to a printer or reading from a file.

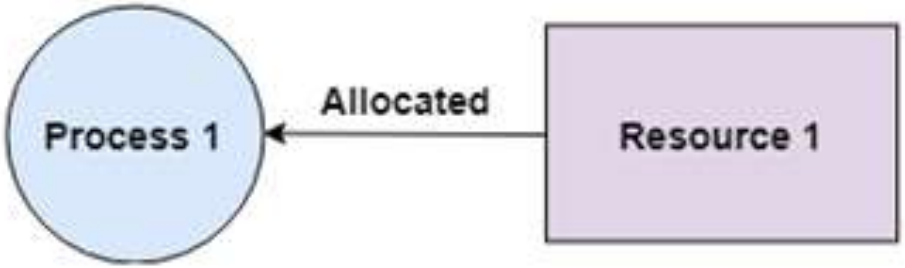
Release –

The process relinquishes the resource, allowing it to be used by other processes.

Deadlock Characterisation : There are four conditions that must be met in order to achieve deadlock as follows.

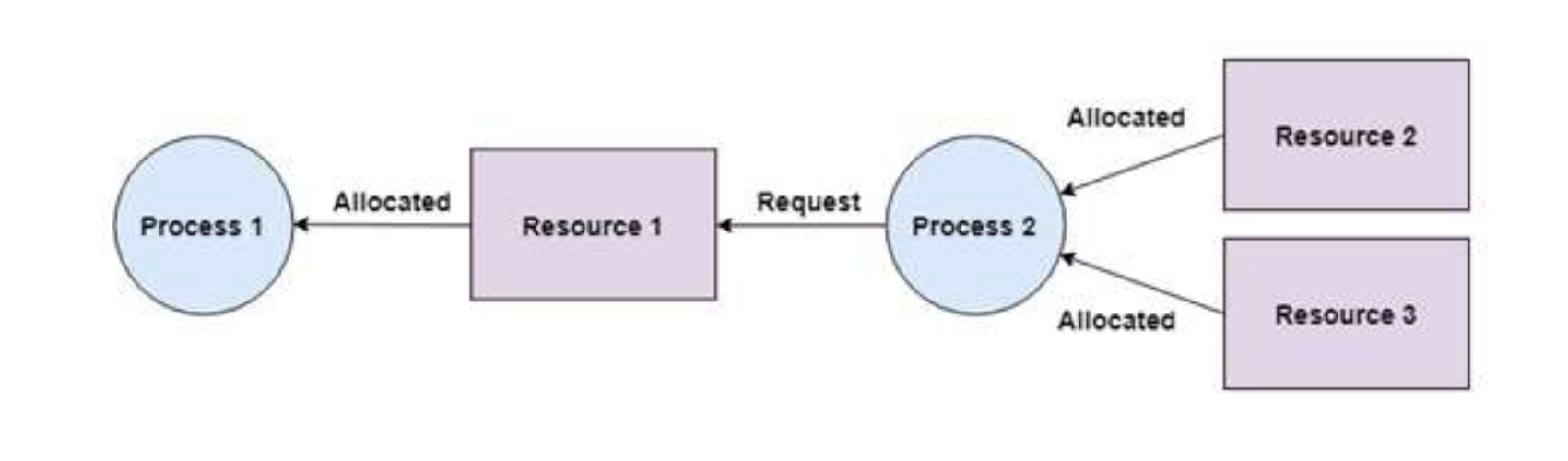
Mutual Exclusion

Only one process can use a resource at any given time i.e. the resources are non-sharable.



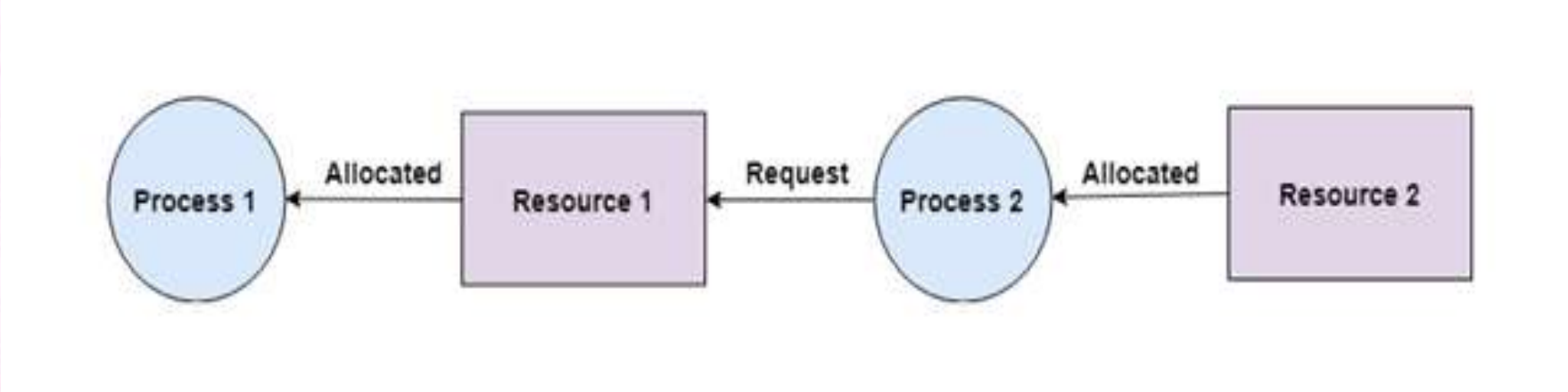
Hold and Wait

A process must hold at least one resource while also waiting for at least one resource that another process is currently holding.



No pre-emption

Once a process holds a resource (i.e. after its request is granted), that resource cannot be taken away from that process until the process voluntarily releases it



Methods for Handling Deadlock

- Deadlock ignorance
- Deadlock prevention

  - make all 4 necessary condition false
- Deadlock avoidance
- Deadlock detection and prevention

Deadlock prevention

Deadlock prevention is a way to make sure that a deadlock, where processes get stuck waiting for each other and can't move forward, never happens in a computer system. It sets up rules to manage how resources are used, so processes can't get into a situation where they are stuck.

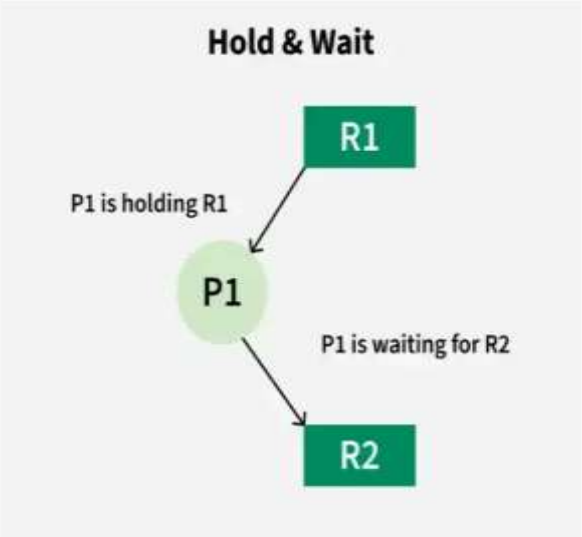
Eliminate Mutual Exclusion

- Mutual exclusion is a condition in which only one process can use a resource at a time.
- When multiple processes compete for the same resource, mutual exclusion can lead to a deadlock.
- If one process has a resource, another process must wait.
- This condition, combined with other factors, can cause a system to stop working.

Eliminate Hold and Wait

There are two ways to eliminate hold and wait:

- **By eliminating wait**: The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.
- For Example, Process1 declares in advance that it requires both Resource1 and Resource2.
- **By eliminating hold**: The process has to release all resources it is currently holding before making a new request.
  For Example: Process1 must release Resource2 and Resource3 before requesting Resource1.



Hold & Wait

Eliminate No Preemption

Preemption is temporarily interrupting an executing task and later resuming it. Two ways to eliminate No Preemption:

- **Processes must release resources voluntarily**: A process should only give up resources it holds when it completes its task or no longer needs them.
- **Avoid partial allocation**: Allocate all required resources to a process at once before it begins execution. If not all resources are available, the process must wait.

Eliminate circular wait

To eliminate circular wait for deadlock prevention, we can use **order on resource allocation**.

- Assign a unique number to each resource.
- Processes can only request resources in an increasing order of their numbers.

A **Resource Allocation Graph (RAG)** is a visual way to understand how **resources** are assigned in an **operating system**. Instead of using only tables to show which resources are allocated, requested, or available, the RAG uses nodes and edges to clearly illustrate relationships between **processes** and their required resources.

RAGs primarily help in detecting deadlocks by visually representing the relationships between processes and resources, making it easier to identify potential deadlock conditions.

A Resource Allocation Graph shows which resources are held by which processes and which processes are waiting for specific types of resources, including how many resources are available, how many are allocated, and what each process needs.

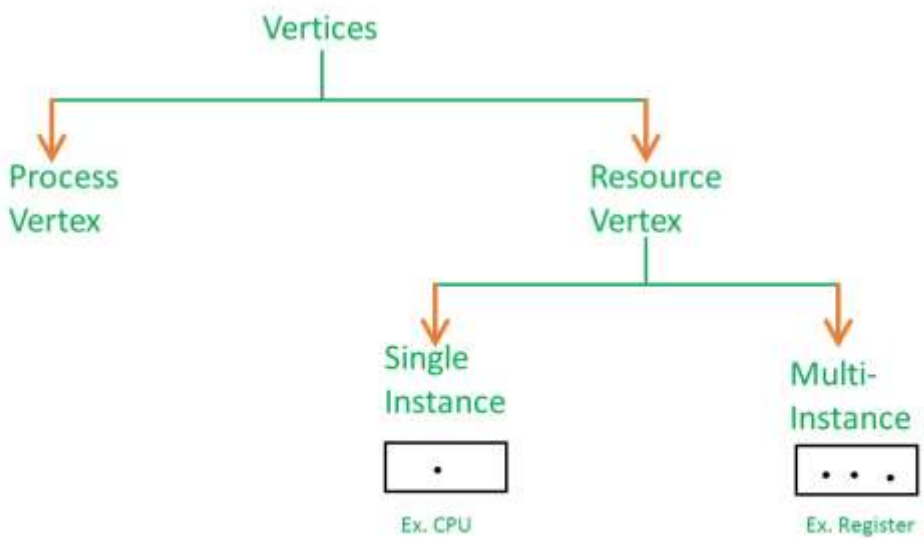
It's a straightforward tool for visualizing potential deadlocks in an operating system since the graph can make deadlock situations more obvious than a table might.

Types of Vertices in RAG

In a Resource Allocation Graph, there are two types of vertices

1. Process Vertex: Every process will be represented as a process vertex. Generally, the process will be represented with a circle.

2. Resource Vertex: Every resource will be represented as a resource vertex. It is also two types:

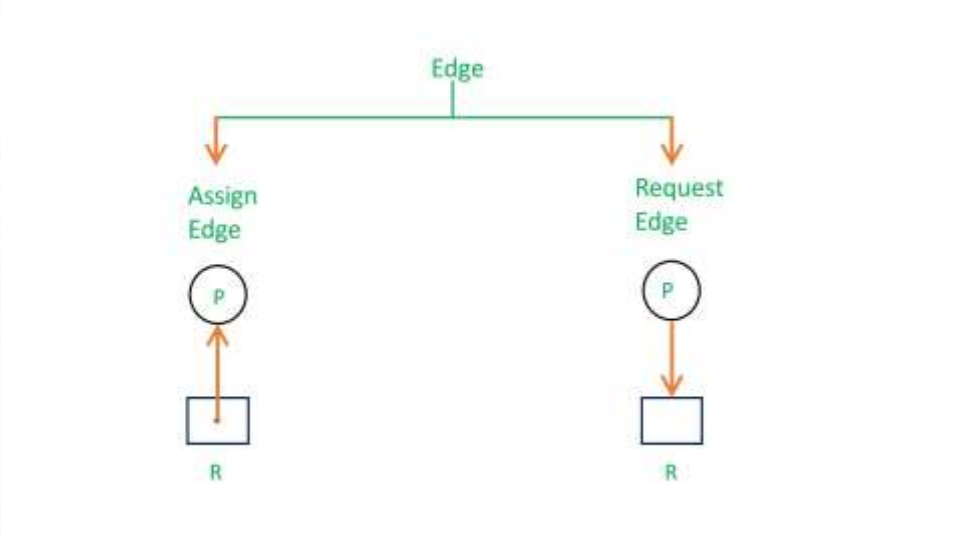


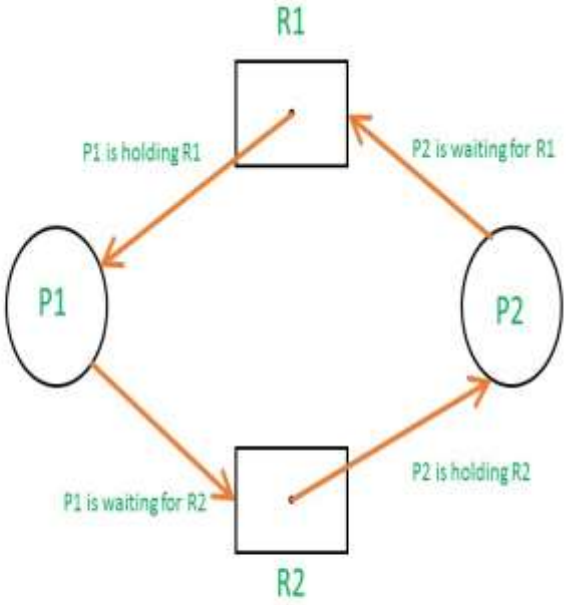
Types of Edges in RAG

Now coming to the edges of RAG. **There are two types of edges in RAG:**

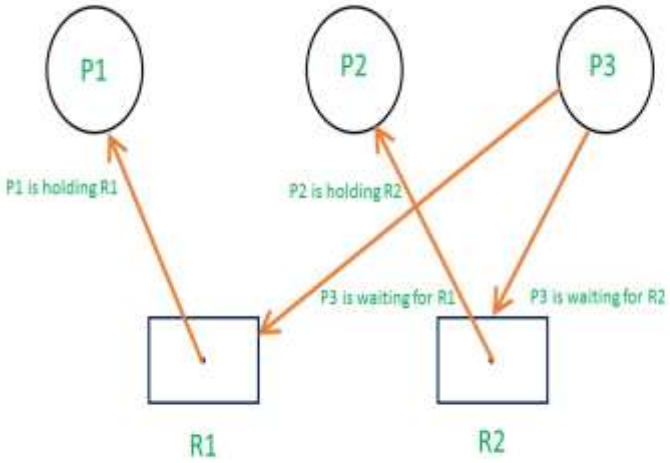
- **Assign Edge:** If you already assign a resource to a process then it is called Assign edge. This is shown by an arrow from the resource vertex to the process vertex.

- **Request Edge**: A request edge represents that a process is currently requesting a resource. This is shown by an arrow from the process vertex to the resource vertex.

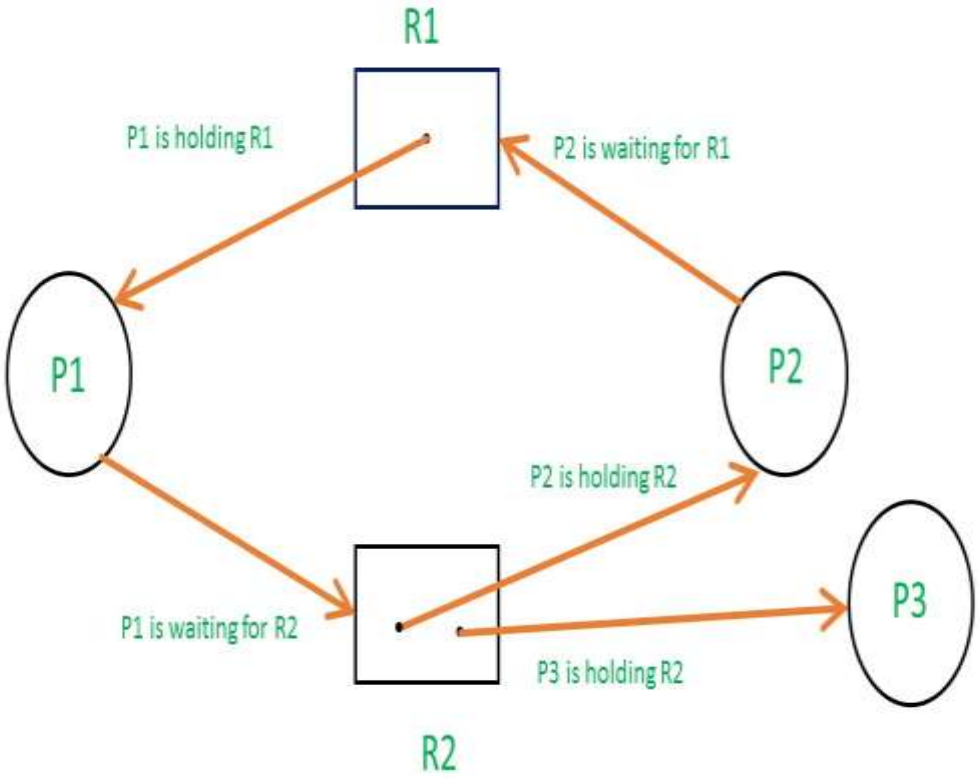




SINGLE INSTANCE RESOURCE TYPE WITH DEADLOCK



SINGLE INSTANCE RESOURCE TYPE WITHOUT DEADLOCK

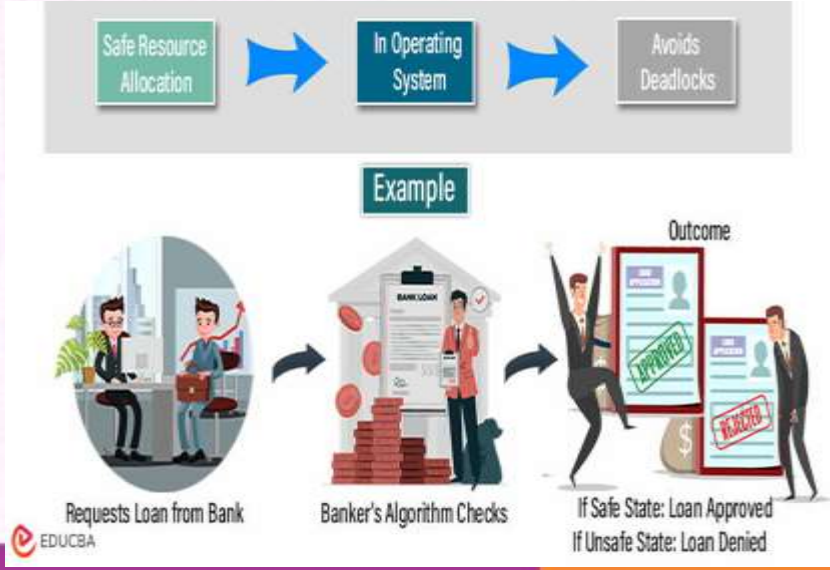


MULTI INSTANCES WITHOUT DEADLOCK

Deadlock Avoidance

The banker's algorithm is a well-known algorithm for preventing deadlocks. Its name comes from the analogy of a bank loan scenario, where a banker makes sure that the bank does not go bankrupt even if all customers apply for a loan at the same time, similar to an operating system that ensures that resource allocation does not cause deadlocks.

The core idea of the banker's algorithm is that the system performs a deadlock prevention check before the process starts to execute, ensuring that the resources allocated to the process are still in a secure state while meeting the maximum resource requirements of the process. This check is done by simulating the state of the system after resource allocation, and if the system is able to enter a safe state, the resource is allowed; Conversely, if an unsafe state is caused, resources are not allocated to avoid deadlocks.



Deadlock Avoidance

Banker's Algorithm

Bankers' Algorithm is a **resource allocation and deadlock avoidance algorithm** that tests all resource requests made by processes. It checks for the **safe state**, and if granting a request keeps the system in safe state, the request is allowed. However, if no safe state exists, the request is denied.

Inputs to Banker's Algorithm

- Max needs of resources by each process.
- Currently, allocated resources by each process.
- Max free available resources in the system.

Safety Algorithm

The algorithm for finding out whether or not a system is in a safe state can be described as follows:

1. Let Work and Finish be vectors of length ‘m’ and ‘n’ respectively.
Initialize: Work = Available
Finish[i] = false; for i=1, 2, 3, 4….n
2. Find an i such that both
2.1 Finish[i] = false
2.2 $Need_i$ <= Work
if no such i exists goto step (4)
3. Work = Work + Allocation[i]
Finish[i] = true
goto step (2)
4. if Finish [i] = true for all i
then the system is in a safe state

Total A=10, B=5 and C=7 Resources

Process	Allocation			Max Need			Remaining Need			Current Availabe		
	A	B	C	A	B	C	A	B	C	A	B	C
~~P1~~	~~0~~	~~1~~	~~0~~	~~7~~	~~5~~	~~3~~	~~7~~	~~4~~	~~3~~	3	3	2
~~P2~~	~~2~~	~~0~~	~~0~~	~~3~~	~~2~~	~~2~~	~~1~~	~~2~~	~~2~~	5	3	2
~~P3~~	~~3~~	~~0~~	~~2~~	~~9~~	~~0~~	~~2~~	~~6~~	~~0~~	~~0~~	7	4	3
~~P4~~	~~2~~	~~1~~	~~1~~	~~4~~	~~2~~	~~2~~	~~2~~	~~1~~	~~1~~	7	4	5
~~P5~~	~~0~~	~~0~~	~~2~~	~~5~~	~~3~~	~~3~~	~~5~~	~~3~~	~~1~~	7	5	5
	7	2	5							10	5	7

Safe Sequence will be: P2 → P4 → P5 → P1 → P3

Deadlock Recovery

Process Termination

Abort all deadlocked processes: This method terminates all the processes involved in the deadlock. It is the simplest method but can lead to loss of all progress made by these processes.

Abort one process at a time: In this case, the system selectively terminates one process involved in the deadlock cycle to break the circular wait. The terminated process is usually the one that has made the least progress or consumes the fewest resources.

Rollback of processes: The system can roll back some processes to a safe state, undoing some of their operations, and freeing up the resources held by them.

Deadlock Recovery

- **Resource Preemption:** Resources are preempted from the processes involved in the deadlock, and preempted resources are allocated to other processes so that there is a possibility of recovering the system from the deadlock. In this case, the system goes into starvation.

Inter-Process Communication or IPC is a mechanism that allows processes to communicate. It helps processes synchronize their activities, share information, and avoid conflicts while accessing shared resources.

Inter-Process Communication refers to the techniques and methods that allow processes to exchange data and coordinate their activities.

The two fundamental models of Inter Process Communication are:

- Shared Memory
- Message Passing

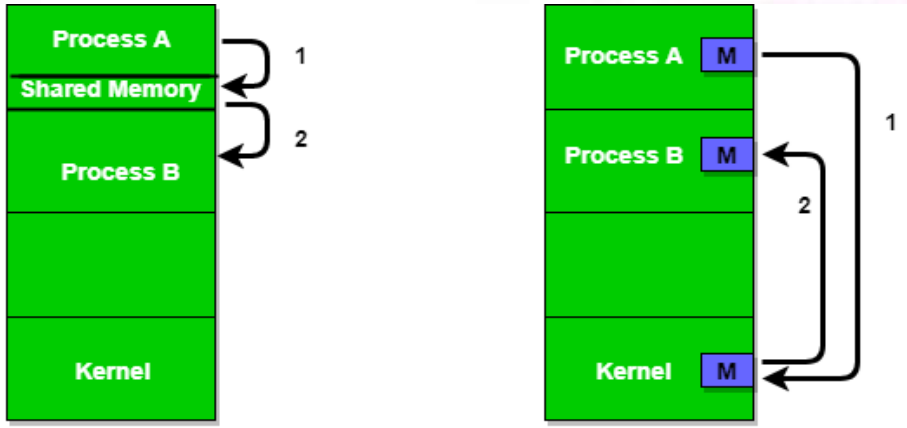


Figure 1 - Shared Memory and Message Passing

Different methods of Inter process Communication (IPC) are as follows:

- **Pipe : Unidirectional communication**
- **Sockets: Network communication**
- **Shared memory :** Generate information for certain computations or shared resources and keep it as record in a shared memory
- **Semaphore:** used for controlling access to shared resources
- **Message Passing:** This allows messages to be passed between processes using either a single queue or several message queue. This is managed by system kernel these messages are coordinated using an API.

Communication in Client Server : In an Operating System, Client Server Communication refers to the exchange of data and Services among multiple machines or processes.

In Client client-server communication System one process or machine acts as a client requesting a service or data, and Another machine or process acts like a server for providing those Services or Data to the client machine.

This Communication model is widely used for exchanging data among various computing environments like Distributed Systems, Internet Applications, and Networking Application communication. The communication between Server and Client takes place with different Protocols and mechanisms.

Different Ways of Client-Server Communication

In Client Server Communication we can use different ways.

Sockets Mechanism Remote Procedure Call Message Passing Inter-process Communication Distributed File Systems

This Photo by Unknown Author is licensed under CC BY